



Actividad 3

Nombre Alumno: Abish Abril Santana García

Materia: Estructura de datos

Nombre Profesor: Rojas Perea, Adalberto
Emmanuel



Descripción

El objetivo de esta actividad es que implementes y apliques tus conocimientos de recursividad y algoritmos de divide y vencerás mediante la resolución de problemas clásicos en programación. Se te proporcionará una serie de problemas que deberás resolver utilizando técnicas recursivas, recursividad indirecta y algoritmos de backtracking. Desarrollarás programas en Java para cada problema, documentando tu proceso y razonamiento.

Objetivo

- Aplicar conceptos de recursividad directa e indirecta.
- Implementar algoritmos de backtracking y divide y vencerás.
- Desarrollar la habilidad para resolver problemas complejos mediante la descomposición en subproblemas.

Requerimientos de la actividad, ejercicio o práctica

- Java Development Kit (JDK) y un IDE como IntelliJ IDEA, Eclipse o Netbeans.
- Comprensión básica de los conceptos de recursividad y algoritmos de divide y vencerás.
- Lectura previa de los temas.

Instrucciones

1. Primer problema: Serie de Fibonacci recursiva. Implementa una función recursiva en Java que calcule el enésimo número en la serie de Fibonacci. Asegúrate de incluir tanto el caso base como el caso recursivo.

1. Método Recursivo Fibonacci

```
public static int fibonacci(int n) {
```

Explicación: n- La posición en la serie que queremos calcular y retorna el valor correspondiente a la posición n en la serie

2. Casos

```
if (n == 0) {  
    return 0;  
}  
if (n == 1) {  
    return 1;  
}
```

Explicación: Caso 1, si n=0, retorna 0 (primer número de la serie) pero si n =1 retorna 1 (segundo número de la serie).

3. Caso Recursivo

```
return fibonacci(n - 1) + fibonacci(n - 2);
```

Explicación: Aquí aplica la definición de Fibonacci, $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$, se llama así misma dos veces con valores menores.

4. Entrada de datos

```
Scanner scanner = new Scanner(System.in);  
System.out.print("Ingrese la posición que desea calcular: ");  
int n = scanner.nextInt();
```

Explicación: Se crea el objeto y se solicita al usuario que ingrese una posición numérica.

5. Validar

```
if (n < 0) {  
    System.out.println("Error: La posición debe ser un número no negativ  
o.");  
}
```

Explicación: Verificar que el numero sea positivo o cero.

6. Calculo y Resultado

```
int resultado = fibonacci(n);  
System.out.println("El número Fibonacci en la posición " + n + " es: " + resultado);
```

Explicación: Se llama a la función recursiva con el valor dado y se muestra

7. Comprobación

```
=== SERIE DE FIBONACCI RECURSIVA ===  
Ingrese la posición que desea calcular: 10  
El número Fibonacci en la posición 10 es: 55  
PS C:\Users\SANTANA\Desktop\Estructura De Datos> |
```

2. Segundo problema: Suma de subconjuntos (Subset Sum). Desarrolla un algoritmo recursivo para determinar si existe un subconjunto de un conjunto dado de enteros que sume un valor objetivo.

1. Método Recursivo Principal

```
public static boolean existeSubconjuntoSuma(int[] conjunto, int n, int sumaObjetivo) {
```

Explicación

- conjunto: Arreglo con los números a evaluar
- n: Cantidad de elementos considerados actualmente
- sumaObjetivo: Valor que queremos alcanzar con la suma

2. Suma Objetivo Cero

```
if (sumaObjetivo == 0) {  
    return true;  
}
```

Explicación

- Si la suma objetivo es 0, siempre existe solución (el subconjunto vacío)
- Este es el caso bueno de la recursión.

3. Sin Elementos Disponibles

```
if (n == 0) {  
    return false;  
}
```

Explicación

- Si no hay más elementos para considerar y la suma no es 0, no existe solución
- Este es el caso de fracaso de la recursión

4. Elementos Grandes

```
if (conjunto[n - 1] > sumaObjetivo) {  
    return existeSubconjuntoSuma(conjunto, n - 1, sumaObjetivo);  
}
```

Explicación

- Si el último elemento es mayor que la suma objetivo, no puede ser incluido
- Se descarta automáticamente y se continúa con los elementos restantes

5. Entrada de Datos

```
Scanner scanner = new Scanner(System.in);  
System.out.print("Ingrese el tamaño del conjunto: ");  
int tamaño = scanner.nextInt();
```

Explicación

- Solicita al usuario el número de elementos del conjunto

6. Lectura de conjuntos

```
int[] conjunto = new int[tamaño];  
for (int i = 0; i < tamaño; i++) {  
    System.out.print("Número " + (i + 1) + ": ");  
    conjunto[i] = scanner.nextInt();  
}
```

Explicación

- Lee cada elemento del conjunto uno por uno

7. Suma Objetivo

```
System.out.print("Ingrese la suma objetivo: ");
int sumaObjetivo = scanner.nextInt();
```

Explicación

- Solicita el valor que se desea alcanzar con la suma

8. Procedimiento y Resultados

```
boolean existe = existeSubconjuntoSuma(conjunto, tamaño, sumaObjetivo);
if (existe) {
    System.out.println("Si existe un subconjunto que suma " + sumaObjetivo);
} else {
    System.out.println("No existe un subconjunto que suma " + sumaObjetivo);
}
```

Explicación

- Llama a la función recursiva y muestra el resultado

```
PS C:\Users\SANTANA\Desktop\Estructura De Datos>
ptionMessages' '-cp' 'C:\Users\SANTANA\AppData\Roa
ra De Datos_8c762411\bin' 'SumaSubconjuntos'
=== SUMA DE SUBCONJUNTOS ===
Ingrese el tamaño del conjunto: 4
Ingrese los 4 números del conjunto:
Número 1: 2
Número 2: 4
Número 3: 6
Número 4: 8
Ingrese la suma objetivo: 10
Sí existe un subconjunto que suma 10
```

3. Tercer problema: Algoritmo de backtracking para el problema del sudoku. Implementa un algoritmo de backtracking que resuelva un Sudoku. El programa debe llenar las celdas vacías del tablero de Sudoku dado.

1. Función puede poner

```
public static boolean puedePoner(int[][] tablero, int fila, int col, int num) {  
    // Mirar la fila  
    for (int i = 0; i < 9; i++) {  
        if (tablero[fila][i] == num) {  
            return false;  
        }  
    }  
}
```

Explicación

Verifica 3 reglas:

- El número no está en la misma **fila**
- El número no está en la misma **columna**
- El número no está en el mismo **cuadrante 3x3**

2. Función Resolver

```
public static boolean resolver(int[][] tablero) {  
    for (buscar vacíos) {  
        for (probar números 1-9) {  
            if (puedePoner) {  
                poner número;  
                if (resolver(resto)) return true;  
                quitar número; // backtrack  
            }  
        }  
        return false;  
    }  
    return true;  
}
```

Explicación

1. **Busca** casillas vacías (0)
2. **Prueba** números del 1 al 9
3. **Verifica** si es válido

4. **Si funciona** → sigue resolviendo
5. **Si no funciona** → retrocede y prueba otro número
6. **Repite** hasta completar todo el tablero

Resultados

```
sudoku:
+---+---+---+
| 53. | .7. | ... |
| 6.. | 195 | ... |
| .98 | ... | .6. |
+---+---+---+
| 8.. | .6. | ..3 |
| 4.. | 8.3 | ..1 |
| 7.. | .2. | ..6 |
+---+---+---+
| .6. | ... | 28. |
| ... | 419 | ..5 |
| ... | .8. | .79 |
+---+---+---+
Calculando...
¡Lo resolví!
+---+---+---+
| 534 | 678 | 912 |
| 672 | 195 | 348 |
| 198 | 342 | 567 |
+---+---+---+
| 859 | 761 | 423 |
| 426 | 853 | 791 |
| 713 | 924 | 856 |
+---+---+---+
| 961 | 537 | 284 |
| 287 | 419 | 635 |
| 345 | 286 | 179 |
+---+---+---+
```